MambaTIG: Fast and Robust Encrypted Traffic Detection Leveraging Selective State-Space Masking

Junbo Jia, Ao Wang, Li Yang, Lu Zhou, *Member, IEEE*, Anyuan Sang, and Huipeng Yang

***Abstract*—The rapid proliferation of encrypted communication strengthens data confidentiality and integrity but also creates blind spots for security monitoring, as adversaries increasingly abuse encrypted channels to conceal malicious activity. Existing approaches typically rely on very large models trained on massive labeled attack datasets, hindering rapid response to novel threats and stable deployment in high-throughput or edge environments; moreover, learning-based methods can suffer performance degradation when attack types, protocol compositions, and deployment environments change. To address these issues, we present MambaTIG, a fast and robust detection framework that couples self-supervised masked graph learning with selective state-space modeling. MambaTIG is pretrained solely on benign traffic to learn discriminative representations from unlabeled data, and it introduces a node-importance–driven masking strategy to improve robustness under distribution shifts; in the graph representation stage, a *Mamba* module replaces attention with linear state-space modeling, efficiently capturing long-range dependencies while reducing inference overhead. Comprehensive evaluations on four public datasets show that MambaTIG achieves competitive and best-or-tied-best mean F1 performance under the standard detection setting. Additional evaluations further show that MambaTIG achieves an average relative F1 improvement of 56.29% across the evaluated cross-task, cross-domain, and cross-scenario distribution-shift settings, and improves adversarial-setting F1 by 37.63% on average compared with the remaining methods. In addition, MambaTIG reduces average inference time by 66.7% compared to the remaining methods.**

***Index Terms*—Encrypted traffic, traffic detection, network security, graph neural network.**

I. Introduction

ENCRYPTED technologies such as Transport Layer Security (TLS), DNS over HTTPS (DoH), and Virtual Private Networks (VPNs) are now pervasive, ensuring data confidentiality and integrity [1]. However, adversaries increasingly exploit encrypted channels to hide malicious activities, creating blind spots for firewalls and intrusion detection systems. Recent reports indicate that over 87% of cyber threats in 2023–2024 were delivered via encrypted traffic [2], highlighting the urgent need for detection techniques tailored to encrypted environments.

Detecting malicious behavior within encrypted channels is substantially more challenging than in plaintext settings. Encryption conceals application-layer payloads, thereby rendering deep packet inspection (DPI) [3] and signature-based intrusion detection [4] largely ineffective. In the absence of payload visibility, early approaches [5], [6], [7] primarily relied on statistical metadata. Although lightweight and readily available, such features provide limited discriminative power and often generalize poorly across different attack types, application scenarios, and deployment environments. More recent studies increasingly leverage deep learning and graph neural networks (GNNs) [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18] to model structural dependencies among flows and capture contextual information, thereby improving detection accuracy. Despite these advances, existing methods still face the following key challenges:

(i) Dependence on labeled data. Supervised methods [5], [6], [7], [16], [19], [20] remain highly sensitive to large-scale annotations of malicious traffic. On the one hand, the collection and labeling of malicious flows are constrained by privacy regulations, data sovereignty, and enterprise governance processes, resulting in high costs, limited coverage, and poor share ability. On the other hand, real-world threats exhibit *long-tailed distributions* and *continuous evolution* (polymorphism, family variants, and zero-day attacks), meaning that historical labels are intrinsically lagging. In deployment, there is often a pronounced *class-space mismatch* and *prior shift* between training and testing, further limiting the practicality of supervised learning approaches.

(ii) Timeliness of detection models. To pursue marginal accuracy gains, many studies introduce deeper stacks and attention modules [5], [6], [15], [16], which leads to severe *parameter and computation inflation*. On long sequences or dense graphs, attention mechanisms incur high complexity and memory pressure; meanwhile, intrusion-prevention pipelines demand *near real-time* responses, which inherently conflict with heavy models. This tension between *complexity* and *deployability* has become a central barrier to practical adoption.

(iii) Robustness to distribution shifts. Due to variations in attack types, application scenarios, protocol compositions, and deployment environments, the statistical distribution of encrypted traffic may differ substantially between training and testing. Existing methods and models [5], [6], [7], [19] often experience

Received 9 January 2026; revised 6 June 2026; accepted 7 July 2026. Date of publication 10 July 2026; date of current version 1 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62572377 and Grant 62302362, in part by the National Natural Science Foundation of China Young Student Basic Research Program under Grant 625B2139, and in part by the Shaanxi Province Natural Science Basic Research Program under Grant 2025SYS-SYSZD-081. *(Corresponding author: Li Yang.)*

Junbo Jia, Ao Wang, Lu Zhou, Anyuan Sang, and Huipeng Yang are with the School of Computer Science and Technology, Xidian University, Xi'an 710071, China.

Li Yang is with the School of Computer Science and Technology, Xidian University, Xi'an 710071, China, and also with the Engineering Research Center of Blockchain Technology Application and Evaluation, Ministry of Education, Xi'an 100816, China (e-mail: yangli@xidian.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3712032

performance degradation when transferred to different tasks, domains, or scenarios, and this lack of adaptability undermines their reliability in practical environments such as enterprise networks and critical infrastructures.

To address these challenges, we propose MambaTIG, a fast and lightweight framework for encrypted malicious traffic detection that reduces dependence on malicious labels, improves computational efficiency, and enhances robustness to distribution shift. First, we design a benign-only representation learning paradigm, in which the model is pretrained exclusively on benign traffic through self-supervised masked graph modeling. This allows the encoder to capture stable communication patterns and intrinsic structural regularities without requiring malicious labels, providing a general solution to open-world encrypted traffic detection.

Second, we integrate the selective state-space model Mamba [21] into graph representation learning. Unlike attention mechanisms, which often incur quadratic complexity and high memory overhead on long sequences or dense graphs, Mamba captures long-range dependencies with linear $O(L)$ complexity. Its selective gating mechanism suppresses redundant channels and highlights informative ones, enabling the encoder to remain expressive without greatly increasing computational cost. By incorporating state-space updates and a learnable gating factor into node updates, the encoder can also better balance current node features and aggregated neighbor information, improving training efficiency and reducing memory overhead.

Third, to improve robustness under distribution shifts, we propose a structure-importance–driven self-supervised pretraining strategy instead of naive masking. Specifically, we design *TIGPageRank* based on the global topology of the traffic interaction graph to estimate node importance, distinguishing core nodes that consistently characterize benign behaviors from peripheral nodes more vulnerable to noise and transient variations. Based on this ranking, we further introduce a curriculum-style masking strategy, in which the masking difficulty is progressively adjusted during training. In particular, the model starts from relatively easier masking conditions and gradually moves to more challenging ones, allowing it to first learn reliable benign interaction patterns and then progressively strengthen perturbations to improve adaptation to distribution shifts. By combining importance-guided masking with curriculum scheduling, the training process becomes more stable than random masking and encourages the encoder to rely on transferable structural cues, producing more consistent and robust embeddings under cross-task, cross-domain, and cross-scenario distribution shifts.

In summary, the main contributions of this paper are as follows:

- *Framework design:* We present MambaTIG, a unified framework for benign-only encrypted malicious traffic detection. It integrates traffic interaction graph construction, graph representation learning, and anomaly detection into an end-to-end pipeline, addressing limited malicious labels and distribution shifts.
- *Efficient representation learning:* We introduce Mamba-based state-space modeling into graph representation learning, enabling efficient long-range dependency modeling with lower computational overhead and inference latency, thereby improving detection efficiency.
- *Shift-aware masking design:* We develop a node-importance estimation method, *TIGPageRank*, together with an importance-ranking–based curriculum masking strategy. This design helps the model learn more stable benign representations and improves robustness under the evaluated distribution-shift settings.
- *Comprehensive evaluation:* Extensive experiments on four representative datasets show that MambaTIG achieves competitive and best-or-tied-best mean F1 performance under the standard detection setting, while maintaining favorable robustness in the evaluated distribution-shift and adversarial-perturbation settings and reducing inference overhead.

The remainder of this paper is organized as follows. Section II reviews the related work on encrypted traffic detection and classification, with an emphasis on supervised, unsupervised, and self-supervised methods, as well as recent studies on robustness under traffic shifts. Section III presents the threat model and assumptions considered in this work. Section IV details the proposed MambaTIG framework, including interaction graph construction, graph representation learning, and anomaly detection. Section V reports the experimental setup and evaluation results, including detection effectiveness, time overhead, robustness under distribution shifts, and ablation studies. Finally, Section VI concludes the paper and discusses future research directions.

II. Related Work

Early malicious-traffic detectors primarily used signature matching [4], [22], [23], [24], i.e., extracting byte sequences and comparing them with known signatures. Such methods fail on encrypted traffic because payloads are inaccessible. In recent years, researchers have increasingly turned to supervised or unsupervised deep models, graph-based approaches, and large-scale pretraining to tackle encrypted malicious traffic. Below, we provide a brief overview of these directions.

A. Supervised Learning Methods

Supervised approaches learn from labeled data. Early studies mainly extracted flow statistics and then applied classical classifiers. For example, Gu et al. [29] used Naïve Bayes for feature embedding followed by SVM; Anderson [30] used session-level statistics and TLS/SSL metadata with Random Forest; and Barut et al. [31] combined flow statistics and TLS metadata with SVM (RBF) and Random Forest.

Recent deep-learning methods employ models such as CNNs and Transformers to extract temporal, spatial, and interaction cues from raw inputs. DeepPacket [5] applies 1D-CNNs to raw packets; FS-Net [32] adds a reconstruction mechanism with a GRU autoencoder for packet-length sequences; ATVITSC [33] renders session "images" and fuses a ViT with a temporal branch; ECNet [34] performs multi-view fusion with confidence-based decisions; MTSecurity [16] combines a

TABLE I
COMPARISON WITH REPRESENTATIVE BASELINES AND RECENT RELATED STUDIES

Method	Granularity	Input	Learning paradigm	Benign-only	Unknown threat	Shift-aware	Core mechanism
DeepPacket [5]	packet	raw packets	supervised	No	No	No	CNN
ET-Bert [6]	packet/session	bytes/datagrams	supervised	No	No	No	Transformer
GraphDApp [14]	graph	interaction graph	supervised	No	No	No	GNN
Net-Mamba [25]	flow/session	traffic sequence	supervised/few-shot	No	Partial	Partial	Mamba
ContraMTD [15]	graph	graph + behavior cues	unsupervised	No	Yes	Yes	Contrastive GNN
Trident [26]	flow	flow profiles	unsupervised	No	Yes	Partial	One-class + clustering
Di Monda et al. [27]	session	multimodal app traffic	few-shot	No	Partial	Yes	Meta-learning
TCG-IDS [28]	graph	temporal traffic graph	self-supervised	No	Partial	Yes	Contrastive GNN
MambaTIG (ours)	graph	traffic interaction graph	self-supervised + unsupervised detection	Yes	Yes	Yes	GIN + Mamba

raw-byte Transformer with an interaction-graph GNN; TFE-GNN [19] builds a PMI-based byte graph with cross-gated GNN fusion; ET-Bert [6] pretrains a BERT-style Transformer on unlabeled datagrams for downstream fine-tuning; GraphDApp [14] models traffic interaction relationships via GNNs; and Net-Mamba [25] replaces Transformers with a Mamba state-space backbone for efficient inference and few-shot transfer.

Recent supervised studies have further considered label scarcity and distribution shifts in encrypted traffic classification. MetaMRE [35] formulates the task as few-shot learning and proposes a multi-task representation-enhanced meta-learning framework to adapt with only a few labeled samples. Di Monda et al. [27] study the impact of shifts in encrypted mobile-app traffic on multimodal few-shot learning, showing that app updates, unseen applications, and traffic shifts can substantially affect classification performance. Earlier deep-learning-based IDS work [36] also emphasized that evolving network behavior and rapidly changing attacks require more adaptive deep models for traffic detection and intrusion analysis.

B. Unsupervised and Self-Supervised Learning Methods

Supervised encrypted-traffic detectors depend on labels and often generalize poorly to unseen threats. In contrast, unsupervised methods fit a benign baseline and flag deviations. Kitsune [37] uses an online ensemble of micro autoencoders with an aggregator autoencoder for reconstruction-based scoring; CPS-GUARD [38] reconstructs session features and introduces an outlier-aware adaptive threshold for CPS/IoT shifts; ET-Net [39] learns hierarchical multi-resolution sequence embeddings with a GMM; Ido Nevat et al. [40] model temporally correlated traffic via Markov chains and apply a composite-hypothesis GLRT without labels; FlowPrint [41] derives temporal fingerprints from encrypted mobile traffic for semi-supervised app identification; Trident [26] decomposes unknown-traffic detection into one-class subtasks for fine-grained, class-incremental discovery; and TrafCL [42] employs contrastive self-supervision with session augmentations and multi-view features to learn robust representations for encrypted malicious-traffic detection.

Graph representations, which preserve interaction patterns and improve generalization, are also used in unsupervised or label-free detection. AGAE [43] builds attributed flow-interaction graphs and trains an attribute graph autoencoder with reconstruction error as the anomaly score; HyperVision [44] constructs compact interaction graphs and derives structural statistics for clustering and scoring; ContraMTD [15] uses contrastive learning to align local behavioral and global interaction-graph representations for label-free detection with strong cross-scenario robustness.

Recent label-efficient studies have further explored robust traffic representation learning under evolving network conditions. TCG-IDS [28] proposes a self-supervised intrusion detection framework based on temporal contrastive graph learning, showing that graph-based self-supervised modeling can improve robustness and detection capability without large-scale labeled training data.

C. Positioning of MambaTIG

Table I positions MambaTIG against representative encrypted-traffic and intrusion-detection methods along seven practical axes. *Granularity* specifies the traffic unit modeled by each method, such as packets, flows, sessions, or graphs. *Input* identifies the observable encrypted-traffic information used for learning. *Learning paradigm* distinguishes supervised, unsupervised, self-supervised, and few-shot settings. *Benign-only* indicates whether the method can be trained without malicious samples. *Unknown threat* reflects whether the method is designed to detect previously unseen malicious behaviors. *Shift-aware* denotes whether distribution shifts are explicitly considered, and *Core mechanism* summarizes the main modeling technique. These axes clarify the deployability-oriented differences between MambaTIG and existing methods in terms of label dependence, open-world detection capability, shift robustness, and computational design.

MambaTIG is related to two main lines of work: Mamba-based traffic modeling and graph-based encrypted traffic analysis. Compared with Net-Mamba [25], which applies Mamba to traffic sequences, MambaTIG introduces Mamba into traffic interaction graph representation learning, so that long-range dependencies are modeled over interaction structures rather than sequences alone.

MambaTIG is also related to graph-based methods such as GraphDApp [14], MTSecurity [16], and ContraMTD [15]. GraphDApp and MTSecurity are supervised and rely on malicious labels, while ContraMTD uses contrastive graph representation learning without state-space modeling. By comparison, MambaTIG combines traffic interaction graphs with a Mamba-enhanced encoder and an importance-guided masking strategy, and adopts self-supervised pretraining on benign traffic followed by anomaly detection.

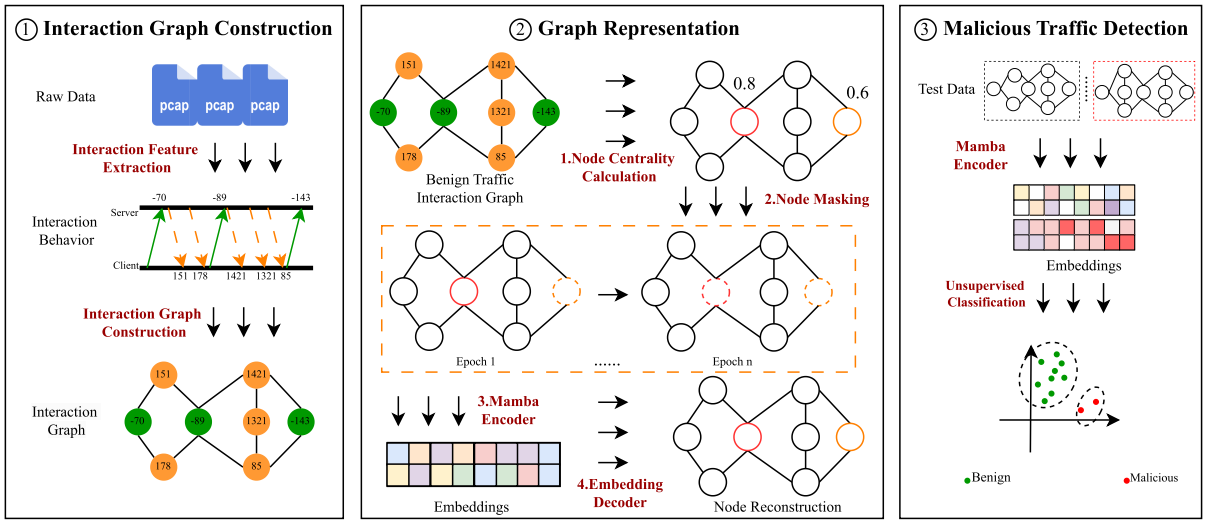


Fig. 1. The overall structure of MambaTIG. MambaTIG comprises interaction graph construction, graph representation, and detection. Raw traffic is converted to an interaction graph, encoded by a masked Mamba into structure-aware embeddings, which are then clustered for unsupervised malicious traffic detection.

In addition, recent studies have considered more challenging deployment settings, such as few-shot adaptation under traffic shifts, robust self-supervised graph learning, and unknown traffic detection. MetaMRE [35], Di Monda et al. [27], TCG-IDS [28], and Trident [26] reflect this trend, while related surveys highlight the importance of robustness to distribution shifts [45]. In this context, MambaTIG is designed for a benign-only setting and focuses on encrypted traffic detection under DDoS-oriented cross-task, application/protocol-domain, and IoT-oriented cross-scenario distribution shifts.

III. Threat Model and Assumptions

The threat model and basic assumptions considered in this work are as follows. We consider a passive detector deployed at a network observation point. The detector cannot access decrypted payload contents and can only observe flow-level metadata, including the five-tuple, packet lengths, packet directions, temporal order, and burst-level interaction patterns, from which traffic interaction graphs are constructed. Following the benign-only setting, we assume that only benign traffic is available during training. Benign samples are used for self-supervised pretraining, reference embedding construction, and threshold calibration, whereas malicious traffic is unavailable during training and appears only at test time.

We consider an open-world deployment scenario in which the detector may encounter distribution shifts after deployment. Such shifts may arise from temporal evolution, scenario changes, application updates, protocol implementation differences, and changes in network environments or user behaviors, as also discussed in recent studies on shifted encrypted traffic classification [27]. Therefore, the threat considered in this work is not restricted to a single fixed data distribution, but includes cross-task, cross-domain, and cross-scenario distribution shifts that may alter the observable characteristics of both benign and malicious traffic.

In addition, we also consider a stronger evasion setting in which an attacker attempts to preserve the malicious functionality of a flow while making its observable characteristics closer to benign traffic. In practice, such manipulations are more likely to be introduced through locally controllable traffic features, especially timing-related characteristics such as inter-packet delays, burst timing patterns, limited padding, or slight packet reordering, which is consistent with the adversarial traffic reshaping setting adopted in TANTRA [46]. By contrast, we do not assume that the attacker can arbitrarily alter the complete global interaction structure of a malicious flow, since overly strong perturbations may violate protocol semantics, disrupt end-to-end functionality, degrade communication quality, or even cause the malicious session itself to fail.

IV. Method

A. Method Overview

Fig. 1 overviews MambaTIG's three modules interaction graph construction, graph representation, and detection aimed at learning benign interaction patterns for unsupervised encrypted-malicious traffic detection while mitigating missing malicious priors and distribution shifts. Training uses only benign PCAPs: traffic is converted to interaction graphs, and a Mamba-based encoder with node-importance masking focuses on salient nodes to capture richer structural/attribute cues. At inference, new traffic is graphed, embedded by the trained Mamba [21] encoder, and flagged via an unsupervised distance-based detector against the benign embedding set.

B. Interaction Graph Construction

The construction of the interaction graph aims to transform raw traffic data into graph-structured data enriched with interaction features. We describe this process from two aspects, namely interaction feature extraction and graph construction, and explain why the interaction graph is advantageous.

1) Interaction Feature Extraction: For interaction feature extraction, we mainly use packet length, direction, and related attributes to characterize interactions between communication entities. During communication, traffic can be collected by tools such as Wireshark. To reduce the impact of irrelevant packets, we first partition the collected traffic by flow. Each flow is defined as a sequence of packets sharing the same five-tuple, i.e., source/destination IP addresses, source/destination ports, and protocol. In this paper, a flow P with n packets is represented as a packet-length vector $P = (p_1, p_2, \ldots, p_n)$, where p_i denotes the signed length of the i-th packet, with negative values for upstream packets and positive values for downstream packets. The choice of n is discussed in the experimental section.

In addition, requests and server responses occur discretely during entity interaction, resulting in intermittent and unstable traffic. Consecutive packets transmitted in the same direction often exhibit strong sequential dependencies. To better characterize flow interaction behavior [14], [47], we group consecutive packets with the same direction into a burst. Accordingly, after feature extraction, the raw flow is transformed into

$$P = \{(p_1), (p_2, p_3, \ldots, p_i), \ldots, (p_j, \ldots, p_k), \ldots, (p_n)\},$$

where $(p_j, \ldots, p_k)$ denotes a burst composed of a contiguous subsequence of packets transmitted in the same direction.

2) Graph Construction: In constructing the Traffic Interaction Graph (TIG), we follow prior studies on Traffic Interaction Graphs [14], [20]. We adopt the same TIG construction to maintain fairness and enable apples-to-apples comparisons with existing TIG-based methods, so that performance differences can be attributed to the proposed learning and detection components rather than changes in graph construction. The definition is as follows:

A *Traffic Interaction Graph* is a triplet $\text{TIG} = (V, E, X)$. Each vertex $v \in V$ stores a signed, nonzero integer whose sign encodes packet direction and magnitude encodes packet length. The edge set E contains (i) *intra-burst* edges linking consecutive packets within the same burst and (ii) *inter-burst* edges linking the first/last packets of adjacent bursts. X is the set of admissible nonzero integers whose absolute values are bounded by the sum of the packet-header length and the *Maximum Transmission Unit* (MTU).

C. Graph Representation

The graph representation stage aims to learn embeddings of benign traffic interaction graphs from self-constructed supervision signals. To this end, we incorporate Mamba blocks into the encoder to capture long-range dependencies with lower computational cost, enabling efficient and accurate representation learning. During training, a strategy-driven masking mechanism and robust optimization process encourage the model to focus on key node attributes and connectivity patterns, thereby improving the fidelity and expressiveness of benign traffic representations. We describe this stage from two aspects: model architecture and training process.

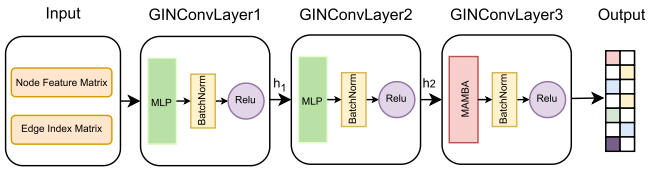


Fig. 2. Architecture of the GIN encoder with Mamba blocks.

1) Model Architecture: The graph representation module consists of an encoder and a decoder. During training, both are optimized using self-constructed supervision signals. During detection, the pre-trained encoder is used to embed the flow interaction graph. The encoder contains three GIN [48] convolutional layers, while the decoder consists of a single GIN convolutional layer.

Encoder: As shown in Fig. 2, the encoder takes the node feature matrix and edge index matrix as input, and outputs the embedding of the traffic interaction graph. The first two layers are MLP-based [49] graph convolutional layers that aggregate neighbor information and update node representations, where a two-layer MLP is used for nonlinear feature mapping. During aggregation and updating, we adopt sum aggregation, since mean aggregation may make different neighborhoods overly similar and reduce distinguishability, while max aggregation ignores overall neighbor information and only preserves the largest feature, which may weaken discriminative ability. Accordingly, the node update in the first two convolutional layers is defined as follows:

$$h_v^{(k)} = \text{MLP}^{(k)}\left((1+\epsilon)h_v^{(k-1)} + \sum_{u \in N(v)} h_u^{(k-1)}\right) \quad (1)$$

Here, $h_v^{(k)}$ denotes the representation of node v at the k-th layer, $N(v)$ is the neighbor set of v, ϵ is a learnable parameter controlling the contribution of the self-loop term, and $\text{MLP}^{(k)}$ performs feature mapping at the k-th layer.

After the MLP-based update, BatchNorm and an activation function are applied to improve stability and expressiveness. BatchNorm alleviates scale variation across layers and maintains consistency across batches, while ReLU introduces nonlinearity and enhances representational capacity.

In the third convolutional layer, we replace the MLP with a Mamba block. Mamba models long-range dependencies through a state-space model with a simpler structure and lower training cost.

Since Mamba is more suitable for high-level than low-level feature extraction, we place it in the final convolutional layer. The first two layers mainly capture local neighborhood information, where long-range dependency modeling is less critical. After two GINConv layers, node features already encode neighborhood context, making the final layer more suitable for modeling deeper global patterns.

By incorporating Mamba's state-space model (SSM) into the convolutional layer, we introduce an SSM-based feature update together with a learnable gating mechanism for dynamically selecting neighbor information. The updated computation is

defined as

$$g_v = \sigma(W_g h_v^{(k-1)} + b_g) \tag{2}$$

$$h_v^{(k)} = (1 - g_v) h_v^{(k-1)} + g_v \,\mathrm{SSM}\left((1+\epsilon) h_v^{(k-1)} + \sum_{u \in N(v)} h_u^{(k-1)}\right) \tag{3}$$

where g_v is a gating factor balancing the current node feature and the aggregated neighbor information. When $g_v = 0$, the previous feature $h_v^{(k-1)}$ is retained; when $g_v = 1$, the node representation is fully updated from the SSM-transformed aggregated feature. Here, SSM denotes the Mamba computation module, which follows the state-space equation.

Decoder: The decoder is designed to reconstruct the *masked interaction structure* rather than raw node attributes. Given the node embeddings z_v produced by the encoder, we adopt a lightweight one-layer GIN-style decoder to (i) estimate edge existence scores for candidate node pairs via $D_\omega(z_u, z_v)$ and (ii) predict node degrees via $g_\phi(z_v)$. During pretraining, the decoder is optimized with an edge-level contrastive reconstruction loss L_{con} and a degree regression loss L_{deg} (see Section IV-C2 and (11)–(16)). Together, these objectives encourage the encoder to learn structure-aware embeddings while keeping the decoder lightweight.

In the decoder design, to avoid the increased complexity and over-smoothing caused by stacking multiple GIN layers for structure reconstruction, we use a single-layer GIN convolutional decoder. This design reduces potential training instability and computational overhead from deeper GIN layers. Moreover, excessive aggregation across multiple layers may make node representations overly similar, leading to over-smoothing. The function of the decoder is expressed as follows:

$$H' = \mathrm{GINConv}(H) \tag{4}$$

where:

- H represents the encoded node embeddings.
- GINConv aggregates neighborhood information together with the node's own features to generate H', thereby improving reconstruction accuracy.

2) Model Training: In MambaTIG, we adopt a graph representation learning approach with a strategy-based masking mechanism for model training. Below, we describe the masking strategy and training method to show how MambaTIG efficiently and accurately represents benign traffic interaction graphs.

Masking Strategy: Similar to prior graph contrastive learning studies [50], [51], our structured masking strategy replaces the attributes of selected nodes with a dedicated mask embedding x_{mask}, thereby suppressing attribute-level cues while preserving essential structural signals. Unlike existing methods, we keep edges unmasked because edges in traffic–interaction graphs (TIGs) encode key relational information. Most existing approaches adopt random node masking, which lacks structural awareness; as a result, the model may repeatedly mask low-information or semantically redundant nodes, weakening self-supervised signals and degrading representation quality. Therefore, selecting masked nodes based on structural salience is important.

To this end, we propose **TIGPageRank**, a weighted personalized PageRank tailored to TIGs for scoring and ranking maskable nodes, so that masking is guided by structural salience rather than random perturbation. The key intuition is that traffic interactions naturally organize into bursts, whose boundaries often align with protocol-phase transitions and client/server turn-taking; thus, boundary events and direction switches serve as stable and interpretable structural anchors. We encode this domain structure in two complementary components. First, we assign intra-/inter-burst edge weights according to (5), allowing the random walk to distinguish within-phase relations from cross-phase transitions. Second, we define a personalization prior in (6) that explicitly emphasizes boundary and direction-switch nodes, so that the stationary importance is concentrated on information-rich regions of the graph. Given the resulting transition probabilities in (7), the damped PageRank iteration in (8) is executed until the convergence criterion in (9) is satisfied; the final scores are normalized via (10) to improve comparability across graphs of different sizes and densities. In implementation, we adopt standard PageRank settings to reduce unnecessary hyperparameter tuning, retaining only the TIG-specific weights $\beta, \lambda_b, \lambda_d$ as controllable knobs, and further study these three hyperparameters in our experiments.

$$W(q,w) = \begin{cases} 1, & \mathrm{bid}(q) = \mathrm{bid}(w), \\ \beta, & \mathrm{bid}(q) \neq \mathrm{bid}(w). \end{cases} \tag{5}$$

$$\pi(w) = \frac{1 + \lambda_b \,\mathrm{boundary}(w) + \lambda_d \,\mathrm{ds}(w)}{\sum_{u \in U} \left(1 + \lambda_b \,\mathrm{boundary}(u) + \lambda_d \,\mathrm{ds}(u)\right)}. \tag{6}$$

$$P(q \to w) = \begin{cases} \frac{W(q,w)}{\sum_{k \in \mathrm{Out}(q)} W(q,k)}, & \mathrm{Out}(q) \neq \emptyset, \\ \pi(w), & \mathrm{Out}(q) = \emptyset. \end{cases} \tag{7}$$

$$R^{(t+1)}(w) = (1 - d_{pr})\,\pi(w) + d_{pr} \sum_{q \in U} R^{(t)}(q)\, P(q \to w). \tag{8}$$

$$\|R^{(t+1)} - R^{(t)}\|_\infty \leq \varepsilon_{\text{PR}}. \tag{9}$$

$$\tilde{R}(w) = \frac{R(w) - \mu_R}{\sigma_R + \varepsilon}. \tag{10}$$

In (5)–(10), $bid(w)$ denotes the burst identifier of node w, and $dir(w)$ denotes the packet direction associated with node w. The indicator function $boundary(w)$ equals 1 if node w is the first or last packet in its burst, and 0 otherwise. Similarly, $ds(w)$ equals 1 if the packet direction of node w differs from that of its predecessor, and 0 otherwise. For a node q, $Out(q)$ denotes the set of its outgoing neighbors in the traffic interaction graph. The parameter β controls the relative weight of inter-burst edges, while λ_b and λ_d control the strengths of the boundary prior and direction-switch prior, respectively. In the PageRank iteration, dpr is the damping factor, $R^{(t)}(w)$ is the importance score of node w at iteration t, and ε_{PR} is the convergence tolerance

Algorithm 1: TIGPageRank.

1: **Input:** $H = (U, E)$; bid(w); dir(w); $\beta, \lambda_b, \lambda_d$
2: **Output:** $\tilde{R}(w)$ (or ranking $\sigma = \text{argsort}(\tilde{R})$)
3: **Defaults:** $d_{pr} \leftarrow 0.85$; $K_{\max} \leftarrow 100$; $\varepsilon_{\text{PR}} \leftarrow 10^{-6}$; $\varepsilon \leftarrow 10^{-8}$
4: **for** $w \in U$ **do**
5: boundary$(w) \leftarrow \mathbf{1}[w$ is first/last in its burst$]$
6: ds$(w) \leftarrow \mathbf{1}[\text{dir}(w) \neq \text{dir}(\text{pre}(w))]$
7: **end for**
8: **for** $(q \to w) \in E$ **do**
9: $W(q, w) \leftarrow$ (5)
10: **end for**
11: **for** $w \in U$ **do**
12: $\pi(w) \leftarrow$ (6)
13: $R^{(0)}(w) \leftarrow \pi(w)$
14: **end for**
15: **for** $t = 0$ to $K_{\max} - 1$ **do**
16: compute $P(q \to w)$ by (7)
17: update $R^{(t+1)}(w)$ by (8)
18: **if** $\|R^{(t+1)} - R^{(t)}\|_\infty \leq \varepsilon_{\text{PR}}$ **then**
19: **break**
20: **end if**
21: **end for**
22: **for** $w \in U$ **do**
23: $\tilde{R}(w) \leftarrow$ (10)
24: **end for**
25: **return** $\tilde{R}$ (**or** $\sigma = \text{argsort}(\tilde{R})$)

used in (9). In (10), μ_R and σ_R denote the mean and standard deviation of the node-importance scores within the current graph, and ε is a small positive constant introduced for numerical stability.

Algorithm 1 summarizes the computation of TIGPageRank. Specifically, lines 4–7 identify two types of structurally salient nodes, namely burst-boundary nodes and direction-switch nodes, which are later used to define the personalization prior. Lines 8–10 assign edge weights according to (5), so that intra-burst and inter-burst transitions are distinguished during the random walk. Lines 11–14 compute the prior distribution in (6) and initialize the PageRank scores. Lines 15–21 perform the iterative importance propagation using (7)–(9) until convergence. Finally, lines 22–24 normalize the resulting scores according to (10), yielding the final node-importance ranking used by the masking strategy.

Training Method: During training, MambaTIG derives its self-supervision from discrepancies between masked nodes and their reconstructions, jointly leveraging node-attribute information. To avoid the computational burden and overfitting risk commonly associated with deep MLP decoders, we adopt a minimalist decoder: a single GIN-style convolutional layer whose update function is a one-layer MLP. This decoder predicts, for each candidate node pair, the probability of an edge, and is optimized so that the aggregate of predicted incident edges around a node matches its degree in the masked graph—thereby easing optimization and stabilizing learning.

Our loss function comprises two terms. The first is an edge-level reconstruction term that evaluates how well the masked graph is recovered at the level of edges. It is defined as follows:

$$\mathcal{L}^{+} = \frac{1}{|\mathcal{E}^{+}|} \sum_{(u,v)\in\mathcal{E}^{+}} \log D_\omega\left(E_u, E_v\right) \tag{11}$$

$$\mathcal{L}^{-} = \frac{1}{|\mathcal{E}^{-}|} \sum_{(u',v')\in\mathcal{E}^{-}} \log\left(1 - D_\omega\left(E_{u'}, E_{v'}\right)\right) \tag{12}$$

$$\mathcal{L}_{\text{con}} = -\left(\mathcal{L}^{+} + \mathcal{L}^{-}\right) \tag{13}$$

where E_u and E_v represent the node representations of nodes u and v generated by the graph encoder, D_ω denotes the edge decoder, and $\mathcal{E}^{+}$ and $\mathcal{E}^{-}$ denote the positive edge set and the sampled negative edge set, respectively. The terms $\mathcal{L}^{+}$ and $\mathcal{L}^{-}$ are the corresponding averaged log-likelihood losses over positive and negative samples. The contrastive reconstruction loss $\mathcal{L}_{\text{con}}$ leverages similar and dissimilar node pairs in the graph as self-supervised signals.

The second component of the objective is the Regression loss. This term penalizes discrepancies between the predicted node degrees and their ground-truth degrees in the masked graph. Concretely, we compute the mean squared error (MSE) at the node level between the predicted and original degrees and average over all nodes. The loss is defined as follows:

$$\mathcal{L}_{\text{deg}} = \frac{1}{|\mathcal{V}|} \sum_{v\in\mathcal{V}} \left\| g_\phi\left(z_v\right) - \deg_{\text{mask}}(v) \right\|_F^2 \tag{14}$$

Where $g_\phi(z_v)$ is the degree decoder, formulated as follows:

$$g_\phi\left(z_v\right) = \text{GINConv}\left(z_v\right) \tag{15}$$

Finally, the loss function we need to minimize is as follows:

$$\mathcal{L} = \mathcal{L}_{\text{con}} + \mathcal{L}_{\text{deg}} \tag{16}$$

To improve optimization stability during pretraining and strengthen the model's ability to represent benign behavior patterns, we propose an importance-guided curriculum structured masking strategy. In this work, *curriculum* refers to a progressive training schedule in which the masking difficulty is gradually increased over time, rather than applying the same masking strength throughout the entire pretraining process. Concretely, training starts with relatively mild masking configurations, allowing the encoder to first capture basic and reliable benign interaction patterns. The masking difficulty is then increased stage by stage, so that the model gradually adapts to harder reconstruction tasks and learns to rely on more transferable structural dependencies. This design reduces early-stage gradient oscillations and unstable convergence caused by overly aggressive masking. Meanwhile, masking locations are selected according to node-importance rankings, turning masking from random perturbation into targeted occlusion of structurally salient information. This encourages the encoder to exploit critical structural cues more effectively and improves sample efficiency. Overall, the strategy enhances the effectiveness and robustness of representation learning without introducing substantial additional

Algorithm 2: Training With Curriculum Structured Masking.

1: **Input:** $\mathcal{D} = \{H_i = (U_i, E_i, X_i)\}_{i=1}^{N}$, mask token x_{mask}, T, p (or p_t), γ, damping d_{pr}, θ, ω, ϕ, optimizer $\mathcal{O}$
2: **Output:** θ, ω, ϕ
3: **for** $i = 1$ to N **do**
4: $I_i(\cdot) \leftarrow \text{TIGPAGERANK}(H_i; \beta, \lambda_b, \lambda_d, d_{pr}, \varepsilon_{\text{PR}})$
5: $\sigma_i \leftarrow \text{argsort}(I_i)$ (ascending)
6: **end for**
7: **for** $t = 1$ to T **do**
8: $p_t \leftarrow p$
9: $\rho_t \leftarrow \begin{cases} 0, & T = 1 \\ (\frac{t-1}{T-1})^{\gamma}, & T > 1 \end{cases}$
10: **for** $i = 1$ to N **do**
11: $n \leftarrow |U_i|, \quad m \leftarrow \min\{n, \max\{0, \lfloor p_t n \rfloor\}\}$
12: **if** $m = 0$ **then**
13: **continue**
14: **end if**
15: $s \leftarrow 1 + \lfloor \rho_t (n - m) \rfloor$
16: $s \leftarrow \min\{s,\ n - m + 1\}$
17: $M_{i,t} \leftarrow \{\sigma_i(s), \sigma_i(s+1), \ldots, \sigma_i(s+m-1)\}$
18: $\hat{X}_{i,t} \leftarrow \text{MASKNODES}(X_i, M_{i,t}, x_{\text{mask}})$
19: $\hat{H}_{i,t} \leftarrow (U_i, E_i, \hat{X}_{i,t})$
20: $Z_{i,t} \leftarrow f_\theta(\hat{H}_{i,t})$
21: $\mathcal{L}_{\text{con}} \leftarrow \text{EDGERECONLOSS}(Z_{i,t}, E_i; \omega)$
22: $\mathcal{L}_{\deg} \leftarrow \text{DEGREGLOSS}(Z_{i,t}, \deg_{\text{mask}}; \phi)$
23: $\mathcal{L} \leftarrow \mathcal{L}_{\text{con}} + \mathcal{L}_{\deg}$
24: $(\theta, \omega, \phi) \leftarrow \mathcal{O}((\theta, \omega, \phi), \nabla\mathcal{L})$
25: **end for**
26: **end for**
27: **return** θ, ω, ϕ

training complexity, making it suitable for handling unseen variations and distribution shifts.

As shown in Algorithm 2, our training procedure operates on a benign-only corpus of traffic–interaction graphs $\mathcal{D} = \{H_i = (U_i, E_i, X_i)\}_{i=1}^{N}$ and a mask token x_{mask}, and adopts a curriculum structured masking strategy guided by node-importance rankings produced by TIGPAGERANK. The inputs include the graph set $\mathcal{D}$, the mask token x_{mask}, the training and masking-schedule hyperparameters, including the base masking rate p, the curriculum length T, and a control factor γ that shapes curriculum progression, as well as the model parameters (θ, ω, ϕ) and an optimizer. The output is the pretrained parameter tuple (θ, ω, ϕ). The algorithm first computes node-importance scores for each graph and derives an ordering over maskable nodes (lines 3–6). At each training epoch $t \in \{1, \ldots, T\}$, it determines the masking schedule and selects a contiguous window on the ranking sequence to form the mask set (lines 7–17), then replaces the corresponding node attributes with x_{mask} to construct a masked graph (lines 18–19). The masked graph is then fed into the encoder to produce node representations, and a lightweight decoder computes the joint self-supervised objective consisting of edge-level reconstruction and degree regression (lines 20–23), which is minimized to update (θ, ω, ϕ) (line 24). By progressively hardening the masking configuration in an importance-aware manner, the proposed training strategy promotes stable optimization and yields robust structural representations under distributional variations.

D. Anomaly Detection

MambaTIG detects previously unseen encrypted malicious traffic by mapping each traffic interaction graph into a graph-level representation space and identifying anomalies based on its distance to reference benign traffic; when malicious references are available, these distances can also be used. This relies on the assumption that traffic graphs or flows with similar behavioral patterns are close in the representation space. During training, a strategy-driven self-supervised masked graph learning mechanism is used to learn stable benign representations, enabling the encoder to capture robust interaction patterns. As a result, the learned space better separates benign and malicious behaviors during inference, improving anomaly-detection accuracy.

In real-world scenarios, unknown-traffic detection must also address efficiency challenges caused by massive data volumes. To improve scalability, we employ Locality-Sensitive Hashing (LSH) [52] for approximate nearest neighbor search. For a target embedding x, a much larger distance to its k-nearest neighbors than that of normal samples may indicate a potential anomaly. LSH maps similar samples to the same hash bucket with high probability and dissimilar ones to different buckets, allowing queries to be restricted to the same bucket rather than the entire dataset.

The LSH-based detection process includes three steps: constructing the LSH index, performing approximate nearest neighbor search, and computing the anomaly score. First, we build the LSH index for benign reference embeddings generated by the graph representation model. Since cosine similarity is invariant to vector scaling, we adopt random hyperplane hashing as the locality-sensitive hash function for traffic feature graph embeddings. Given an embedding x, a single hash function is defined as follows:

$$h(x) = \text{sign}(a \cdot x) \tag{17}$$

where a is a random vector sampled from the standard normal distribution $\mathcal{N}(0, 1)$. Specifically, $h(x) = 1$ if $a \cdot x > 0$, and $h(x) = 0$ otherwise. This hashing scheme maps data points to one side of a randomly chosen hyperplane, thereby preserving directional similarity in the embedding space.

To improve retrieval recall, we build L independent hash tables. In the ℓ-th hash table, we concatenate m hash functions to form a composite hash function:

$$g_\ell(x) = [h_{\ell,1}(x), h_{\ell,2}(x), \ldots, h_{\ell,m}(x)], \quad \ell = 1, 2, \ldots, L \tag{18}$$

For each benign embedding x_i in the reference set, we compute its bucket identifier $g_\ell(x_i)$ for every hash table and insert x_i into the corresponding bucket. As a result, each bucket stores a subset of embeddings likely to be similar under cosine similarity,

enabling efficient candidate filtering without scanning the entire reference set.

Second, we perform approximate nearest neighbor search for an unknown traffic embedding x. We compute $\{g_\ell(x)\}_{\ell=1}^{L}$ and retrieve candidate embeddings from the buckets indexed by these identifiers across all hash tables. The final candidate set is obtained by merging candidates from all tables:

$$C(x) = \bigcup_{\ell=1}^{L} C_\ell(x) \tag{19}$$

where $C_\ell(x)$ denotes the set of embeddings stored in the bucket indexed by $g_\ell(x)$ in the ℓ-th hash table. We then compute the distance between x and each candidate $y \in C(x)$, and select the k-nearest neighbors $N_k(x)$ for anomaly estimation. Since the hashing scheme is aligned with cosine similarity, we use cosine distance for neighbor ranking:

$$d(x, y) = 1 - \cos(x, y) \tag{20}$$

Third, we compute the anomaly score based on neighborhood distances. If the unknown embedding x is consistent with benign traffic in the learned representation space, it should lie close to benign reference embeddings and yield small neighbor distances. In contrast, embeddings corresponding to unknown malicious traffic are expected to deviate from the benign manifold, resulting in larger distances to their nearest neighbors. Therefore, we define the anomaly score as the average cosine distance to the k-nearest neighbors:

$$s(x) = \frac{1}{k} \sum_{y \in N_k(x)} d(x, y) \tag{21}$$

We determine an anomaly threshold τ from the distribution of scores on benign validation data, and classify x as anomalous when $s(x) > \tau$. This LSH-based retrieval strategy substantially reduces the computational overhead of nearest neighbor search while preserving the discriminative capability of the learned graph representations for unknown encrypted malicious traffic detection.

V. Evaluation

This section evaluates the effectiveness of MambaTIG as a detector for encrypted malicious traffic. First, we describe the experimental setup and the evaluation datasets. Next, we compare MambaTIG with state-of-the-art methods and provide a comprehensive assessment in terms of accuracy, recall, F1 score, runtime efficiency, robustness to distribution shifts, and adversarial robustness. We then analyze the contribution of each component, with particular emphasis on the proposed node-selection algorithm (TIGPageRank) and the training strategy. Finally, we discuss the model's storage footprint and examine how hyperparameters, alternative unsupervised detection algorithms, and architectural variants affect overall detection accuracy.

A. Experimental Setup

In the experimental setup, we mainly introduce the dataset used in the experiment, the experimental environment, and the experimental scheme.

a) Datasets: To ensure a fair evaluation of the effectiveness of MambaTIG, the experimental datasets were selected from those commonly used in existing encrypted malicious traffic detection studies, including USTC-TFC2016 [53], CICIDS2017 [54], CICDDoS2019 [55], and the IoT-23 Dataset [56]. These datasets were collected from real-world networks and encompass both normal and malicious traffic commonly observed in IoT and internet environments.

- *USTC-TFC2016 [53]:* Released by USTC, this dataset contains 10 types of normal traffic and 10 types of malicious traffic, and is widely used for traffic classification and malicious traffic detection.
- *CICIDS2017 [54]:* Provided by CIC, this dataset includes normal traffic and 16 attack types, and is a widely used benchmark for intrusion detection research.
- *CICDDoS2019 [55]:* This dataset focuses on multiple types of DDoS traffic and is commonly used to evaluate DDoS detection methods.
- *IoT-23 [56]:* IoT-23 contains both normal and malicious IoT traffic and is designed for malicious traffic detection in IoT environments.

Following the benign-only unsupervised setting of MambaTIG, we split the benign traffic into three disjoint parts with a ratio of 70/15/15 for training, validation, and testing, respectively. We do not perform any additional processing on the traffic contents of these datasets; instead, we only read and extract the features required for constructing the traffic interaction graph. To reduce the impact of randomness, we repeat the experiments 10 times under different random splits and report the mean and standard deviation of the results. The training split is used for representation learning and to build the benign reference set for nearest-neighbor retrieval. The validation split contains only benign traffic and is used to calibrate the detection threshold τ. Specifically, we set τ to the $(1 - \varepsilon_{\text{fp}})$ quantile of anomaly scores on the benign validation set with $\varepsilon_{\text{fp}} = 0.1\%$, thereby targeting a 0.1% false-positive rate on benign traffic. The test split is used only for final evaluation. All malicious traffic is excluded from training and validation and is used only in the test phase to report detection performance together with the benign test split.

b) Parameter Settings: The main hyperparameters used in MambaTIG are summarized in Table II. For model training, we set the embedding dimension to $d = 512$, the masking ratio to $p = 0.7$, and the learning rate to $l = 0.0025$, and optimize the model using AdamW with a batch size of 64. For TIGPageRank, we use $\beta = 1.5$, $\lambda_b = 1.0$, and $\lambda_d = 2.0$, while the remaining PageRank parameters follow the default settings $d_{pr} = 0.85$, $K_{max} = 100$, $\epsilon_{PR} = 10^{-6}$, and $\epsilon = 10^{-8}$. For the curriculum structured masking strategy, we set the curriculum length to $T = 100$ and the progression factor to $\gamma = 2.0$.

c) Experimental Environment: The experimental environment consists of Ubuntu 22.04.4 LTS (64-bit) as the operating system, with 64 GB of memory, an Intel Xeon(R) Bronze 3106 CPU @ 1.70 GHz, and an NVIDIA A100 GPU.

TABLE II
HYPERPARAMETER SETTINGS FOR MAMBATIG

Parameter	Meaning	Value
d	Embedding dimension	512
p	Masking ratio	0.7
l	Learning rate	0.0025
Optimizer	Training optimizer	AdamW
Batch size	Graphs per mini-batch	64
n	Max packets	50
β	TIGPageRank inter-burst weight	1.5
λ_b	Boundary bias	1.0
λ_d	Direction-switch bias	2.0
d_{pr}	PageRank damping factor	0.85
K_{max}	PageRank max iterations	100
ϵ_{PR}	PageRank tolerance	10^{-6}
ϵ	Stability constant	10^{-8}
T	Training epochs / curriculum length	100
γ	Curriculum progression factor	2.0
m	LSH hash functions per table	11
L	Number of LSH tables	30
k	Nearest neighbors in LSH	20
ϵ_{fp}	Validation false-positive rate	0.1%

d) Experimental Scheme: To comprehensively evaluate the effectiveness of MambaTIG, we selected DeepPacket [5], TFE-GNN [19], ET-Bert [6], GraphDApp [14], MTSecurity [16], Net-Mamba [25], Trident [26], and ContraMTD [15] as baseline methods for comparison. These methods cover traditional feature engineering methods, deep learning methods, neural network-based methods, and Mamba-based methods. For methods with publicly available source code, we used their official implementations; for methods without publicly available source code, we adopted reliable third-party re-implementations. To ensure the fairness and reliability of the experimental results, all compared methods were evaluated under their optimal parameter settings through multiple repeated runs, and the results were used for comparative analysis.

e) Evaluation metric clarification: Unless otherwise specified, "improvement" refers to relative improvement (RI) over a baseline, rather than absolute percentage-point gain:

$$RI(\%) = \frac{F1_{\text{ours}} - F1_{\text{base}}}{F1_{\text{base}}} \times 100\%, \tag{22}$$

where $F1_{\text{base}}$ and $F1_{\text{ours}}$ denote the F1-scores of the baseline and the proposed method, respectively. When multiple baselines are compared, we report the average RI across all baselines and the maximum RI among them.

B. Evaluation of Detection Performance

1) Effectiveness: We evaluate the effectiveness of MambaTIG as a malicious traffic detector on four widely used benchmark datasets. As shown in Table III, MambaTIG achieves competitive performance that is on par with strong baselines under the standard detection setting, attaining the best or tied-best F1-scores across all datasets, namely **98.84%**, **99.39%** (tied), **99.49%**, and **99.61%**, respectively.

This competitive performance can be attributed to the use of a mask-based self-supervised masked graph learning mechanism during the graph representation stage. Unlike GraphDApp and MTSecurity, which also utilize flow interaction graphs but lack contrastive guidance, MambaTIG leverages structure-aware masking to encode the semantic characteristics of benign traffic. This allows the model to focus on stable structural patterns and reduce the influence of noisy or redundant features.

Consequently, MambaTIG can learn discriminative representations of benign samples, which helps form a stable decision boundary during the detection phase. This supports the model in identifying malicious traffic while maintaining comparable detection stability across diverse scenarios.

2) Time Overhead: Time overhead is an important metric for evaluating encrypted malicious traffic detectors. To assess the runtime efficiency of MambaTIG, we conduct experiments on both GPU and CPU platforms. As illustrated in Fig. 3, MambaTIG achieves the lowest time cost among all evaluated methods on both platforms. Specifically, on the CPU, MambaTIG requires only 1.25 milliseconds, representing an 86.1% reduction compared to the slowest baseline ET-Bert (11.45 ms), and an 81.3% reduction relative to the average of all other models. On the GPU, MambaTIG completes inference in just

TABLE III
EVALUATION RESULTS FOR MALICIOUS TRAFFIC DETECTION

Dataset	Scheme	Precision (%)	Recall (%)	F1-Score (%)
USTC-TFC2016	DeepPacket	95.81±0.41%	92.45±0.37%	94.10±0.27%
	ET-Bert	96.54±0.54%	94.23±0.56%	95.37±0.39%
	GraphDApp	89.23±0.61%	83.40±0.74%	86.22±0.49%
	TFE-GNN	96.52±0.57%	93.21±0.58%	94.84±0.41%
	MTSecurity	97.45±0.34%	98.23±0.74%	97.84±0.41%
	Net-Mamba	98.42±0.47%	96.12±0.53%	97.26±0.36%
	ContraMTD	92.45±0.32%	96.32±0.36%	94.35±0.24%
	Trident	98.21±0.50%	97.92±0.48%	98.06±0.35%
	MambaTIG	98.56±0.26%	99.12±0.36%	98.84±0.22%
CICIDS2017	DeepPacket	98.56±0.81%	99.10±1.00%	98.83±0.64%
	ET-Bert	99.45±0.32%	98.52±0.30%	98.98±0.22%
	GraphDApp	98.41±0.27%	92.35±0.36%	95.28±0.23%
	TFE-GNN	97.54±0.69%	98.52±0.78%	98.03±0.52%
	MTSecurity	99.23±0.26%	99.56±0.31%	99.39±0.20%
	Net-Mamba	99.45±0.46%	99.23±0.51%	99.34±0.34%
	ContraMTD	97.52±0.24%	98.23±0.27%	97.87±0.18%
	Trident	97.44±0.44%	96.51±0.46%	96.97±0.32%
	MambaTIG	99.32±0.22%	99.47±0.31%	99.39±0.19%
CICDDoS2019	DeepPacket	95.24±0.84%	98.52±0.78%	96.85±0.57%
	ET-Bert	99.45±0.99%	97.42±0.99%	98.42±0.70%
	GraphDApp	94.23±0.66%	95.23±0.58%	94.73±0.44%
	TFE-GNN	96.82±0.53%	98.36±0.60%	97.58±0.40%
	MTSecurity	97.88±0.67%	99.81±0.59%	98.84±0.45%
	Net-Mamba	98.73±0.39%	99.32±0.50%	99.02±0.32%
	ContraMTD	96.87±0.31%	92.35±0.35%	94.56±0.24%
	Trident	97.54±0.47%	96.23±0.43%	96.88±0.32%
	MambaTIG	99.43±0.33%	99.56±0.41%	99.49±0.26%
IoT-23	DeepPacket	96.23±0.57%	98.70±0.51%	97.45±0.39%
	ET-Bert	95.63±0.34%	93.24±0.48%	94.42±0.30%
	GraphDApp	98.52±0.67%	96.23±0.35%	97.36±0.38%
	TFE-GNN	98.56±0.76%	94.57±0.68%	96.52±0.51%
	MTSecurity	99.81±0.60%	97.85±0.54%	98.82±0.40%
	Net-Mamba	99.01±0.39%	98.52±0.45%	98.76±0.30%
	ContraMTD	84.56±0.44%	91.52±0.50%	87.90±0.33%
	Trident	96.21±0.47%	97.58±0.42%	96.89±0.31%
	MambaTIG	99.42±0.39%	99.81±0.40%	99.61±0.28%

Note: Green shading highlights the best mean for each dataset/metric. Results are reported as mean ± standard deviation over 10 runs.

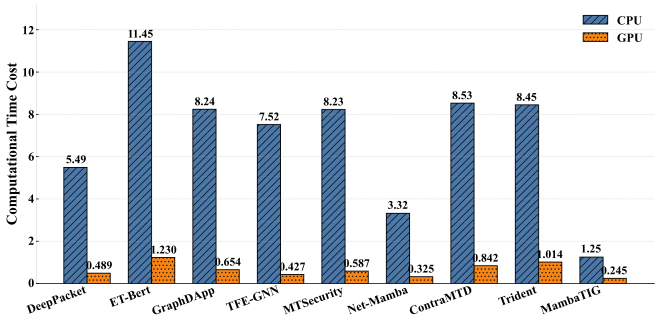


Fig. 3. Time overhead comparison on GPU and CPU.

0.24 milliseconds, achieving an 80.5% reduction compared to ET-Bert (1.23 ms), and reducing the average GPU inference time of the remaining methods by 66.7%.

The efficiency advantage of MambaTIG comes from two aspects. First, the Mamba module introduced in the graph representation stage captures long-range dependencies in traffic interaction graphs through linear state-space modeling, thereby reducing the computational cost and inference latency of representation learning. Second, MambaTIG adopts a lightweight detection pipeline that combines graph representation learning with a non-parametric anomaly detection stage based on Locality-Sensitive Hashing (LSH), avoiding the additional overhead of a heavy end-to-end classifier. Therefore, the efficiency gain of MambaTIG should not be attributed to TIG-based feature modeling alone, but rather to the joint effect of the Mamba module and the overall lightweight detection pipeline.

C. Evaluation Under Distribution Shifts

In traffic detection, the data-generating distribution may shift with changes in attack types, application scenarios, protocol compositions, and deployment environments. To evaluate the robustness of MambaTIG under such distribution shifts, we consider three representative cross-distribution transfer settings: *DDoS-oriented cross-task distribution shift*, *cross-scenario/domain distribution shift*, and *IoT-oriented cross-scenario distribution shift*. Specifically, we first train on *CICIDS2017* and test on *CICDDoS2019* to evaluate the model's transfer performance from a comprehensive intrusion-detection scenario to a DDoS-dominated detection task. We further evaluate the model on *IoT-23* and *USTC-TFC2016* to examine its cross-scenario generalization under different application/protocol compositions and operational environments.

To ensure a fair comparison, all methods are trained only on the source dataset and directly evaluated on the target dataset. For supervised baselines, such as DeepPacket, ET-Bert, Net-Mamba, and MTSecurity, we follow their original supervised learning paradigms and provide both benign and malicious samples from the source dataset for training. All attack categories are mapped to the malicious class, so the task remains binary malicious-traffic detection rather than multi-class attack-type classification. In contrast, MambaTIG uses only benign samples from the source dataset for pretraining and detection-threshold construction. Labels from the target dataset are used only for final evaluation and are not used for training, fine-tuning, threshold selection, or model adaptation for any method.

In the *CICIDS2017*-to-*CICDDoS2019* DDoS-oriented cross-task distribution-shift experiment (see Fig. 4(a)), MambaTIG achieves precision/recall/F1 of **95.37%/97.42%/96.38%**. Compared with the baseline methods, it obtains an average relative F1 improvement of **53.35%**. Flow-statistics-based methods, represented by DEEPPACKET, degrade more noticeably under this cross-task setting, whereas interaction-graph-based methods, including ContraMTD and Trident, remain relatively stable. The advantage of MambaTIG mainly comes from its ability to model global interaction patterns and its structure-aware masking mechanism for exploiting stable traffic structures.

In the *CICIDS2017*-to-*USTC-TFC2016* cross-scenario/domain distribution-shift experiment (see Fig. 4(b)), MambaTIG achieves precision/recall/F1 of **80.21%/79.63%/79.92%**. Its relative F1 improvement over Trident is **12.83%**, and its average relative F1 improvement over all baselines reaches **63.14%**. The performance decrease compared with the *CICDDoS2019* setting is expected, because *USTC-TFC2016* differs from *CICIDS2017* in application composition, traffic style, and collection background, resulting in more pronounced feature-distribution changes.

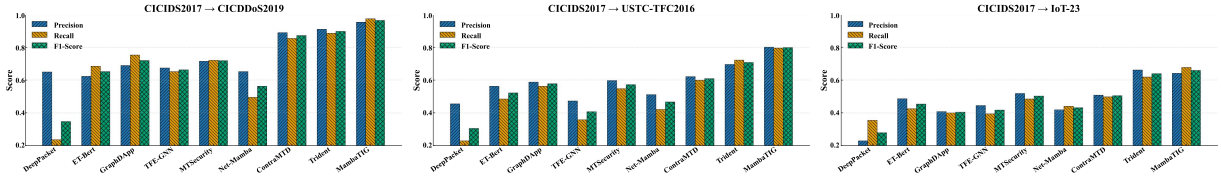
In the *CICIDS2017*-to-*IoT-23* IoT-oriented cross-scenario distribution-shift experiment (see Fig. 4(c)), MambaTIG achieves an average relative F1 improvement of **52.37%** and exceeds the second-best method by **3.17%**. IoT traffic places greater emphasis on protocols such as Telnet, IRC, and DNS, together with device heartbeat and command-and-control patterns, rather than HTTP/HTTPS, FTP, and SSH. As a result, the absolute scores of all methods decrease; nevertheless, MambaTIG maintains favorable detection performance. These results suggest that MambaTIG provides robust generalization under the evaluated cross-task, cross-domain, and cross-scenario distribution-shift settings.

D. Ablation Study

We evaluate the overall impact of MambaTIG on performance. Specifically, we study (i) the effect of introducing the Mamba block on detection accuracy and computational cost and (ii) the influence of the node-importance selection strategy; in Section V-G & Section V-H we further discuss the impact of architectural depth and the role of the unsupervised anomaly-detection algorithm.

1) Impact of the Mamba Block: We conducted experiments under identical conditions for versions with and without the Mamba block. The impact on the MambaTIG framework is shown in the Table IV.

With the introduction of the Mamba block, the model achieves significant improvements in both detection accuracy and computational efficiency across four benchmark datasets, as shown in Table IV. These consistent gains demonstrate that the integration of the Mamba module substantially enhances the model's capability to learn discriminative features from malicious traffic graphs.



(a) CICIDS2017 → CICDDoS2019 (b) CICIDS2017 → USTC-TFC2016 (c) CICIDS2017 → IoT-23

Fig. 4. Detection performance under distribution shifts across three transfer settings.

TABLE IV
THE ABLATION STUDY RESULTS OF THE MAMBA BLOCK

	USTC-TFC2016			CICIDS2017			CICDDoS2019			IoT-23		
	ACC	F1	Time (ms)	ACC	F1	Time (ms)	ACC	F1	Time (ms)	ACC	F1	Time (ms)
GIN	93.22%	92.56%	1.230	92.47%	91.12%	0.658	91.37%	88.74%	0.586	96.43%	94.58%	0.875
GIN+Mamba	98.86%	98.83%	0.823	99.32%	99.39%	0.253	99.21%	99.49%	0.356	99.63%	99.61%	0.421

This performance improvement is primarily attributed to the linear state space modeling capability of Mamba, which enables the model to capture long-range dependencies in graph structures with low computational overhead. Unlike traditional graph neural networks that rely on multi-hop message passing, Mamba propagates structural semantic information globally through dynamic convolution operations, thereby enhancing its capacity to represent complex attack behaviors. For instance, on the IoT-23 dataset, which contains diverse and semantically varied traffic, the integration of Mamba boosts the F1-score from 0.9458 to 0.9961, demonstrating superior generalization ability.

In addition to accuracy gains, Mamba significantly reduces inference latency. On CICIDS2017, inference time decreases from 0.658 ms to 0.253 ms, and on CICDDoS2019, from 0.586 ms to 0.356 ms. This indicates that Mamba features a lightweight architecture and excellent parallelizability, effectively avoiding redundant computation by eliminating deep message aggregation layers. Moreover, its low-pass filtering characteristics help suppress structural noise in the graph, thereby reducing overfitting risks and improving the robustness and stability of anomaly detection.

In summary, the Mamba block not only enhances the model's ability to represent structural semantics in graphs but also achieves an excellent trade-off between detection performance and computational efficiency.

2) Impact of Node Importance Ranking Methods: To assess the impact of node-importance evaluation on strategic masked-graph learning, we conduct experiments under a fixed masking rate using four widely adopted centrality measures—degree centrality, closeness centrality, betweenness centrality, and PageRank—together with our proposed *TIGPageRank*. As shown in Fig. 5, ACC and F1 differ markedly across metrics. Among all methods, *TIGPageRank* consistently delivers the best performance (ACC = 0.9963, F1 = 0.9961), significantly outperforming the alternatives. Unlike local indices that consider only direct connectivity or similar proximal cues, our burst-aware PageRank assigns weighted emphasis to traffic bursts—particularly burst boundaries and direction switches—thereby providing a more comprehensive assessment of each node's influence in the graph. Moreover, the damping-based iterative computation renders the method more robust to structural noise, enhancing generalization. In the context of strategic masking, this increases the likelihood that later training stages expose the model to informative, representative nodes, ultimately enabling more effective learning of semantic structures within the graph.

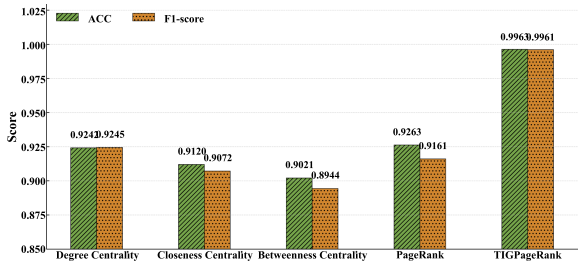


Fig. 5. Comparison of different node importance ranking methods on performance.

TABLE V
IMPACT OF MASKING STRATEGY UNDER DISTRIBUTION SHIFTS

Masking Strategy	Precision	Recall	F1-score
Strategy-driven	95.37%	97.42%	96.38%
Random	90.50%	92.60%	91.50%

3) The Impact of Masking Strategies: We investigate the effect of masking strategies at the graph-representation stage by comparing (i) strategy-driven masking, which selects masked nodes according to a node-importance ranking, and (ii) random masking. As shown in Table V, strategy-driven masking consistently outperforms random masking in terms of precision, recall, and F1-score.

We evaluate both strategies under *distribution shifts*. The results show that strategy-driven masking consistently outperforms random masking in terms of precision, recall, and F1-score.

TABLE VI
PERFORMANCE COMPARISON ON CICIDS2017 UNDER ORIGINAL AND ADVERSARIAL SETTINGS

Method	Original Benign + Original Malicious			Original Benign + Adversarial Malicious		
	Precision (%)	Recall (%)	F1-score (%)	Precision (%)	Recall (%)	F1-score (%)
DeepPacket	98.56	99.1	98.83	10.12	9.80	9.957
ET-Bert	99.45	98.52	98.98	20.04	18.44	19.20
GraphDApp	98.41	92.35	95.28	74.52	78.45	76.43
TFE-GNN	97.54	98.52	98.03	74.43	69.88	72.08
MTSecurity	99.23	99.56	99.39	84.52	82.21	83.34
Net-Mamba	99.45	99.23	99.34	21.05	22.57	21.78
ContraMTD	97.52	98.23	97.87	80.14	79.55	79.84
Trident	97.44	96.51	96.97	74.56	73.51	74.03
MambaTIG	99.32	99.47	99.39	91.87	92.56	92.21

These findings indicate that strategy-driven masking is markedly more effective than random masking: by prioritizing the masking of less informative nodes, it provides a curriculum-like self-supervision signal that strengthens representation learning. Specifically, we estimate node importance via *TIGPageRank*, which helps the model identify globally influential nodes and their associated structures. This importance-guided mechanism keeps high-value nodes visible with higher probability during training, thereby anchoring the main interaction backbone of the graph, while suppressing incidental perturbations introduced by low-value nodes and reducing the model's reliance on noisy and non-stationary details. Overall, strategy-driven masking enables the encoder to learn more stable and semantically consistent interaction representations and to be less sensitive to spurious local patterns, leading to improved detection performance under distribution shifts.

E. Evaluation Under Adversarial Traffic Reshaping

To further evaluate the robustness of MambaTIG against adversarial attacks, we adopt a timing-based adversarial traffic reshaping setting inspired by TANTRA [46]. Specifically, we construct adversarial malicious traffic by applying timing-oriented perturbations to the original malicious flows while preserving their original attack semantics as much as possible.

Based on the CICIDS2017 dataset, we evaluate the models under two test settings: (i) *Original Benign + Original Malicious*, and (ii) *Original Benign + Adversarial Malicious*. The first setting is used as the standard evaluation protocol, while the second is used to assess the robustness of the detector under adversarial perturbations.

As shown in Table VI, all methods suffer performance degradation to different degrees under the adversarial traffic setting, but the extent of degradation varies significantly. Methods that rely more heavily on local sequential patterns, such as DeepPacket, ET-Bert, and Net-Mamba, degrade more severely, whereas methods based on interaction-structure modeling remain relatively more stable. MambaTIG still achieves the best Precision, Recall, and F1-score, indicating stronger robustness against this type of adversarial perturbation. This can be attributed to the fact that MambaTIG mainly exploits packet lengths, directions, and global interaction structures during graph construction and representation learning, rather than directly relying on timestamps or inter-packet delays, which are more easily manipulated by attackers. Therefore, timing-based adversarial reshaping has a relatively limited impact on MambaTIG. However, it should also be noted that the performance of MambaTIG still declines to some extent, suggesting that although timing perturbations are not directly used as model inputs, they can still affect local interaction patterns by altering packet arrival rhythms, and may further interfere with burst segmentation and the overall interaction structure representation, thereby weakening detection performance.

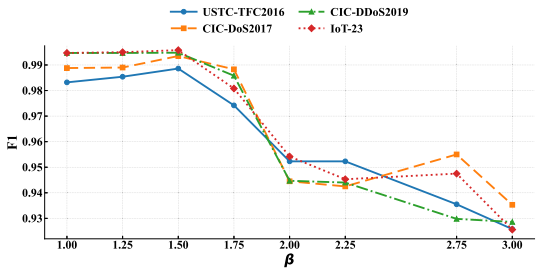


Fig. 6. Sensitivity to the inter-burst edge weight β.

F. Hyperparameter Experiments

We investigate how different hyperparameters affect the performance of MambaTIG, covering two categories: (i) the **TIG-PageRank** hyperparameters β, λ_b, and λ_d, and (ii) the model-training hyperparameters, including the embedding dimension d, the masking rate P, and the learning rate l.

1) TIGPageRank Hyperparameters: In **TIGPageRank**, β controls the relative weight of inter-burst edges in the random walk, thereby balancing within-phase relations versus cross-phase transitions. Meanwhile, λ_b and λ_d jointly modulate the personalization prior that emphasizes burst-boundary and direction-switch nodes, concentrating importance on structurally salient anchors. Since in practice we mainly care about their relative strength, we denote the ratio by $\alpha \triangleq \lambda_d/\lambda_b$, treat α as a single hyperparameter, and discuss the TIG-structure-related hyperparameters β and α.

a) The Inter-burst Edge Weight: Fig. 6 reports the sensitivity to the inter-burst edge weight β. Overall, β exhibits a clear "moderate-is-best" behavior: increasing β from 1 to

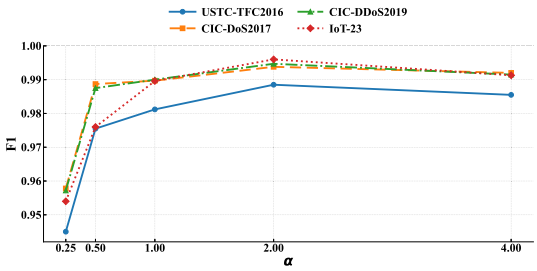


Fig. 7. Sensitivity to the prior-strength ratio α.

1.5 consistently improves (or keeps near-optimal) F1 across all four datasets, and USTC-TFC2016, CIC-DoS2017, and IoT-23 achieve their best results at $\beta = 1.5$. This trend aligns with the role of β in **TIGPageRank**: by upweighting inter-burst edges, the random walk is encouraged to propagate importance across burst boundaries, thereby better highlighting structurally salient transition points that often coincide with protocol-phase changes and client/server turn-taking. In this regime, the ranking becomes more sensitive to phase-level structural cues rather than being dominated by redundant within-burst interactions, leading to more informative masking decisions.

When β is further increased to 2 and beyond, performance drops systematically on all datasets. A larger β makes cross-burst propagation overly aggressive, causing importance mass to diffuse along potentially noisy or incidental transitions and reducing the contrast between truly salient nodes and ordinary nodes. As a result, the importance ranking becomes less discriminative, and the subsequent masking is more likely to obscure suboptimal node sets, degrading the quality of self-supervision. Taken together, these results suggest that setting β slightly above 1 offers a robust balance between capturing stable within-phase relations and emphasizing meaningful cross-phase transitions; we therefore adopt $\beta = 1.5$ as the default in our experiments.

b) The Prior-Strength Ratio: Fig. 7 studies the impact of the prior-strength ratio α on TIGPageRank. We observe a monotonic improvement as α increases from 0.25 to 2, where all four datasets achieve their best or near-best F1, followed by a mild drop at $\alpha = 4$. This behavior is consistent with the role of α in the personalization prior, which controls the relative emphasis on direction-switch versus burst-boundary anchors. When α is small, direction-switch nodes receive insufficient prior mass, and the random-walk restart distribution is dominated by boundary cues; as a result, the importance scores under-utilize turn-taking information and may miss salient bidirectional interaction patterns that are characteristic of benign behaviors. As α increases to a moderate level, direction switches provide a complementary anchor to burst boundaries, yielding a more informative and balanced restart bias that concentrates importance on structurally salient regions. This improves the discriminability of node ranking and, in turn, enables more effective importance-guided masking.

When α becomes too large (e.g., $\alpha = 4$), the prior increasingly over-focuses on direction-switch nodes, which can be more frequent and sometimes noisier than boundary events. This over-bias reduces the contribution of boundary anchors, makes the importance distribution less stable, and can lead to masking decisions that over-emphasize local direction alternations while neglecting phase-level structure. Overall, the results suggest that a moderate ratio provides the best trade-off between exploiting turn-taking cues and preserving robust boundary anchors; we therefore use $\alpha = 2$ as the default in our experiments.

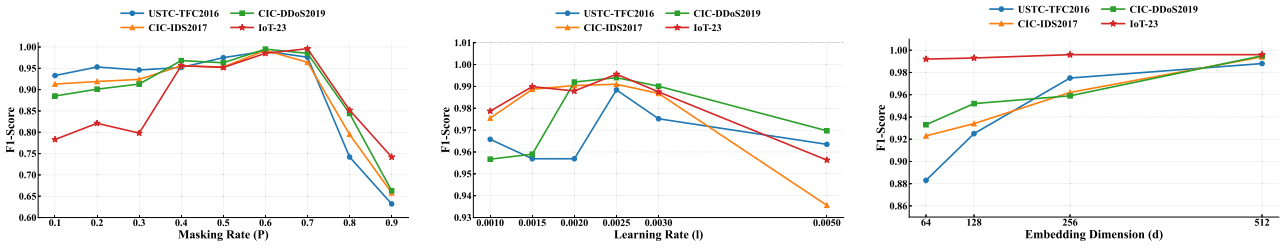
2) Model Hyperparameters: For model training, we analyze the key hyperparameters that govern representation capacity and optimization dynamics, including the embedding dimension d, the masking rate p, and the learning rate l.

a) Impact of Masking Rate: Fig. 8(a) shows the influence of the masking rate p on model performance. The results indicate that the best performance is achieved at $p = 0.7$. When p is small, such as $p \leq 0.2$, only a limited portion of node attributes is corrupted, making the reconstruction objective too easy and the self-supervised signal insufficient, which leads to limited representation learning. In contrast, when p is large, such as $p \geq 0.8$, extensive attribute occlusion removes critical semantic cues and reduces effective context, making it difficult for the model to infer meaningful structure from the remaining graph signals and resulting in a clear performance drop. Overall, $p = 0.7$ provides a favorable trade-off between creating challenging supervision and preserving adequate context for reliable reconstruction.

b) Impact of Learning Rate: Fig. 8(b) presents the sensitivity to the learning rate l. Across datasets, a moderate learning rate in the range of 0.002 to 0.0025 consistently achieves the best and most stable performance. With a smaller learning rate, such as $l = 0.001$, optimization progresses slowly and may not converge sufficiently within a fixed training budget, which can manifest as slower convergence and mild underfitting. With a larger learning rate, such as $l = 0.005$, the update step becomes overly aggressive, which can overshoot good solutions and amplify training instability under masked reconstruction, leading to fluctuating or degraded performance. These results highlight that an appropriate learning rate is critical for balancing convergence speed and training stability.

c) Impact of Embedding Dimension: Fig. 8(c) evaluates the embedding dimension d. Increasing d from 64 to 512 generally improves performance across all datasets, indicating that a higher-dimensional latent space enhances the encoder capacity to capture structural dependencies and semantic interaction patterns in traffic interaction graphs. The improvement is more pronounced on datasets with more complex interaction structures, where additional capacity helps model diverse regimes and long-range dependencies. Beyond $d = 512$, the gain tends to saturate and further increases bring limited benefit while substantially increasing computational cost and memory footprint. Therefore, $d = 512$ offers a practical balance between accuracy and efficiency in our setting.

3) Impact of Packet Count: The packet-count hyperparameter n controls the maximum number of packets used to construct each TIG, and thus determines the graph size and visible interaction context. Increasing n can provide more temporal and burst-level information, but it also increases computation



(a) Masking rate. (b) Learning rate. (c) Embedding dimension.

Fig. 8. Sensitivity analysis of model-training hyperparameters.

TABLE VII
EFFECT OF PACKET COUNT PER FLOW (n) ON EFFICIENCY AND DETECTION PERFORMANCE ON CICDDOS2019

n	Inference time (ms)	Training time (h)	F1 score
10	0.185	2.54	0.7542
20	0.186	3.22	0.8588
30	0.224	3.58	0.9522
40	0.238	4.02	0.9752
50	0.245	4.25	0.9949
60	0.642	5.23	0.9950
70	0.885	5.88	0.9942
80	1.010	6.75	0.9945

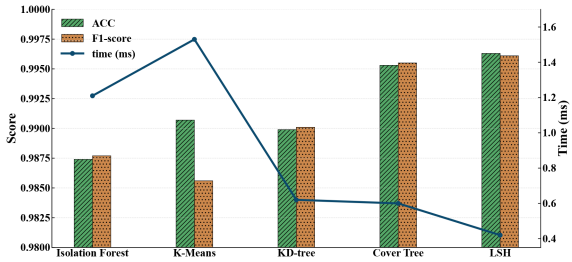


Fig. 9. Performance comparison of detection algorithms.

and may introduce redundant or noisy packets once the main interaction pattern has been captured.

As shown in Table VII, we evaluate the effect of the packet count per flow n on the CICDDoS2019 dataset. When n increases from 10 to 50, the F1 score increases from 0.7542 to 0.9949, indicating that additional packets provide useful temporal and statistical cues for detection. When n is further increased to 60 and beyond, the F1 score largely saturates and fluctuates slightly within 0.9942 to 0.9950, yielding only marginal gains. Meanwhile, the inference time remains low for $n \leq 50$, ranging from 0.185 ms to 0.245 ms, but increases to 0.642 ms at $n = 60$ and continues to rise to 1.010 ms thereafter. The training time also grows with n, increasing from 2.54 h to 6.75 h. Based on this trade-off, we adopt $n = 50$ as the default packet-count setting in all main experiments, as it achieves a high F1 score while maintaining relatively low training and inference costs.

G. Impact of Detection Algorithms

To compare the performance and efficiency of different detection algorithms, we selected five commonly used unsupervised detection or approximate nearest neighbor algorithms: Isolation Forest, K-Means, KD-tree, Cover Tree, and LSH. These methods were evaluated based on three key metrics: ACC, F1-score, and detection time (in milliseconds). As shown in Fig. 9, all algorithms achieved reasonably good classification accuracy, but there were significant differences in terms of time consumption.

Among the methods, LSH achieved the highest F1-score (0.9961) while also exhibiting the fastest detection speed with only 0.421 ms of time overhead. This highlights LSH's excellent trade-off between speed and accuracy. This advantage can be attributed to LSH's ability to construct a set of locality-sensitive hash functions that map high-dimensional vectors into low-dimensional buckets, enabling fast approximate similarity search. This mechanism avoids the performance degradation commonly seen in traditional KD-tree or Cover Tree methods when applied to high-dimensional data, making it particularly well-suited for graph embedding spaces.

Moreover, once the hash index is constructed, LSH enables sublinear-time nearest neighbor search, which is especially critical in real-time malicious traffic detection scenarios. Compared to K-Means or Isolation Forest, which typically require scanning all data points, LSH significantly reduces computational overhead.

Overall, the experimental results confirm that LSH delivers both high detection accuracy and exceptional computational efficiency. These characteristics make it particularly advantageous for building high-dimensional graph embedding-based malicious traffic detection systems, especially in scenarios where real-time performance and resource efficiency are essential.

H. Model Architecture Experiments

We examined how the model architecture influences MambaTIG detection performance, and report the results in Table VIII. With all other factors fixed, increasing the encoder depth from one to three layers yields a monotonic improvement in F1, while increasing the decoder depth generally reduces performance, and this degradation becomes more evident when the encoder is deeper. The best setting is a three-layer encoder paired with a single-layer decoder, which achieves the highest F1 among all configurations.

TABLE VIII
IMPACT OF MODEL DEPTH ON PERFORMANCE

Encoder Layers	Decoder Layers	F1-Score
1	1	0.5963
	2	0.5877
	3	0.5631
2	1	0.7958
	2	0.7235
	3	0.7581
3	1	0.9961
	2	0.9147
	3	0.9025

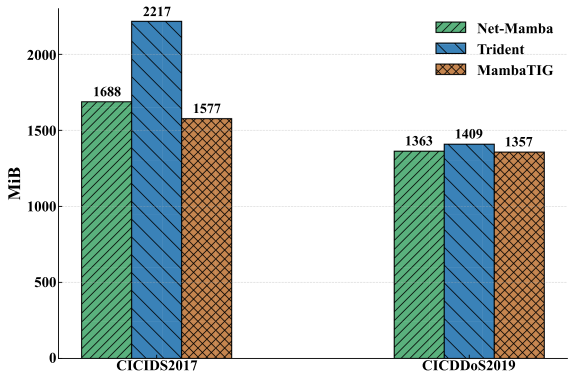


Fig. 10. Experimental results on peak GPU memory consumption (MiB).

This trend suggests that the encoder is the primary source of discriminative representation power. A deeper encoder can aggregate broader multi-hop interaction structure on TIGs and refine higher-level semantics after local neighborhood information has been fused, which improves separability in the learned embedding space. In contrast, an overly deep decoder increases reconstruction capacity and weakens the training pressure on the encoder, since the decoder can rely on its own parameters to explain masked content rather than forcing the encoder to produce informative and transferable representations. As a result, the learned embeddings become less aligned with the downstream detection objective, and training becomes more prone to overfitting and unstable optimization, which ultimately harms generalization and reduces class separability at inference time.

I. GPU Memory Consumption Experiment

As shown in Fig. 10, we evaluate peak GPU memory across CICIDS2017 and CICDDoS2019 and observe consistent savings for the Mamba family. On CICIDS2017, Trident uses 2217 MiB, Net-Mamba 1688 MiB, and MambaTIG 1577 MiB; relative to Trident, Net-Mamba reduces VRAM by 529 MiB or 23.86% and MambaTIG by 640 MiB or 28.87%, with MambaTIG a further 111 MiB or 6.58% below Net-Mamba. On CICDDoS2019, Trident, Net-Mamba, and MambaTIG consume 1409, 1363, and 1357 MiB respectively; the reductions versus Trident are 46 MiB or 3.26% for Net-Mamba and 52 MiB or 3.69% for MambaTIG, with MambaTIG another 6 MiB or 0.44% below Net-Mamba. Averaged over both datasets, the footprints are 1813 MiB for Trident, 1525.5 MiB for Net-Mamba, and 1467 MiB for MambaTIG, implying mean savings of 19.08% against Trident and 3.83% against Net-Mamba for MambaTIG. Stability also favors MambaTIG, whose standard deviation and coefficient of variation are 110 MiB and 7.50% compared with 404 MiB and 22.28% for Trident; the footprint range is 220 MiB for MambaTIG versus 808 MiB for Trident, a reduction of 72.77%. Overall, memory advantages are largest on CICIDS2017, indicating that MambaTIG's linear state-space design yields greater gains on longer or denser sequences and graphs.

VI. LIMITATIONS

Although MambaTIG demonstrates strong accuracy and efficiency across multiple datasets and distribution-shift settings, several limitations remain.

- *Limited evaluation under adversarial settings:* Although the current results indicate robustness to certain interaction-pattern perturbations, this paper does not systematically evaluate performance under stronger adversarial manipulations, such as deliberately crafted burst boundaries, direction switches, or continuously changing attack behaviors. Future work will further analyze and improve the adversarial robustness of MambaTIG.
- *Limited visibility of late-stage malicious behavior:* The current framework considers only the first n packets of each flow, which may reduce sensitivity to attacks whose malicious behavior emerges only in later communication stages. Although this limitation can be partially alleviated by increasing n, doing so also introduces higher computational cost. Future work will further explore adaptive multi-stage observation mechanisms to improve the detection of late-stage malicious behavior while controlling overhead.

VII. CONCLUSION

We propose MambaTIG, a detection framework for unknown encrypted malicious traffic that aims to balance detection performance and computational efficiency. Under the benign-only setting, MambaTIG combines self-supervised masked graph learning with Mamba-based state-space modeling to learn benign traffic-interaction representations for downstream anomaly detection.

In the representation phase, MambaTIG uses strategy-driven masking and linear state-space modeling to capture long-range dependencies in traffic interaction graphs with relatively low overhead. In the detection phase, it employs locality-sensitive hashing (LSH) to compare high-dimensional embeddings efficiently. Experiments on widely used datasets show that MambaTIG achieves competitive detection accuracy and low time overhead, while maintaining favorable robustness in the evaluated distribution-shift and adversarial-perturbation settings.

REFERENCES

[1] Google Online Security Blog, "Google public DNS over HTTPS (DoH) supports RFC 8484 standard," 2019. Accessed: Apr. 9, 2025. [Online]. Available: https://security.googleblog.com/2019/06/google-public-dns-over-https-doh.html

[2] Zscaler ThreatLabz, "Zscaler finds over 87% of cyberthreats hide in encrypted traffic, reinforcing the need for zero trust," 2024. Accessed: Apr. 07, 2025. [Online]. Available: https://www.zscaler.com/press/zscaler-finds-over-87-cyberthreats-hide-encrypted-traffic-reinforcing-need-zero-trust

[3] C. Liu, Y. Pan, A. Chen, and J. Wu, "A DFA with extended character-set for fast deep packet inspection," *IEEE Trans. Comput.*, vol. 63, no. 8, pp. 1925–1937, Aug. 2014.

[4] C. Krügel and T. Toth, "Using decision trees to improve signature-based intrusion detection," in *Proc. Recent Adv. Intrusion Detection, 6th Int. Symp.*, Pittsburgh, PA, USA, 2003, pp. 173–191, doi: 10.1007/978-3-540-45248-5_10.

[5] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft. Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020, doi: 10.1007/s00500-019-04030-2.

[6] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, New York, NY, USA, 2022, pp. 633–642, doi: 10.1145/3485447.3512217.

[7] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "NetMamba: Efficient network traffic classification via pre-training unidirectional Mamba," 2024. [Online]. Available: https://arxiv.org/abs/2405.11449

[8] C. Fan, J. Cui, H. Jin, H. Zhong, I. Bolodurina, and D. He, "Robust intrusion detection system for vehicular networks: A federated learning approach based on representative client selection," *IEEE Trans. Mobile Comput.*, vol. 24, no. 11, pp. 11942–11956, Nov. 2025. [Online]. Available: https://doi.org/10.1109/TMC.2025.3582237

[9] J. Ghadermazi, S. Hore, A. Shah, and N. D. Bastian, "GTAE-IDS: Graph transformer-based autoencoder framework for real-time network intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 4026–4041, 2025.

[10] C. Zha et al., "A-NIDS: Adaptive network intrusion detection system based on clustering and stacked CTGAN," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 3204–3219, 2025.

[11] Y. Wu et al., "Intrusion detection for Internet of Things: An anchor graph clustering approach," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 1965–1980, 2025, doi: 10.1109/TIFS.2025.3539100.

[12] L. Yang et al., "unFlowS: An unsupervised construction scheme of flow spectrum for network traffic detection," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 3330–3345, 2025, doi: 10.1109/TIFS.2025.3550060.

[13] Y. Li, W. Liang, K. Xie, D. Zhang, K. Li, and N. N. Xiong, "Event-Mon: Real-time event-based streaming network monitoring data recovery," *IEEE Trans. Dependable Secur. Comput.*, vol. 22, no. 3, pp. 2413–2429, May/Jun. 2025, doi: 10.1109/TDSC.2024.3519197.

[14] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Accurate decentralized application identification via encrypted traffic analysis using graph neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2367–2380, 2021.

[15] X. Han et al., "ContraMTD: An unsupervised malicious network traffic detection method based on contrastive learning," in *Proc. ACM Web Conf.*, New York, NY, USA, 2024, pp. 1680–1689, doi: 10.1145/3589334.3645479.

[16] J. Yang, X. Jiang, Y. Lei, W. Liang, Z. Ma, and S. Li, "MTSecurity: Privacy-preserving malicious traffic classification using graph neural network and transformer," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3583–3597, Jun. 2024.

[17] Z. Song et al., "I^2RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification," *IEEE Trans. Dependable Secure Comput.*, early access, Feb. 28, 2023, doi: 10.1109/TDSC.2023.3245411.

[18] Y. Yang, C. Kang, G. Gou, Z. Li, and G. Xiong, "TLS/SSL encrypted traffic classification with autoencoder and convolutional neural network," in *Proc. IEEE 20th Int. Conf. High Perform. Comput. Commun., IEEE 16th Int. Conf. Smart City, IEEE 4th Int. Conf. Data Sci. Syst. (HPCC/SmartCity/DSS)*, 2018, pp. 362–369.

[19] H. Zhang et al., "TFE-GNN: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proc. ACM Web Conf.*, New York, NY, USA, 2023, pp. 2066–2075, doi: 10.1145/3543507.3583227.

[20] Y. Li et al., "GraphDDoS: Effective DDoS attack detection using graph neural networks," in *Proc. IEEE 25th Int. Conf. Comput. Supported Cooperative Work Des.*, 2022, pp. 1275–1280.

[21] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.

[22] G. Vigna, W. K. Robertson, and D. Balzarotti, "Testing network-based intrusion detection signatures using mutant exploits," in *Proc. 11th ACM Conf. Comput. Commun. Secur.*, Washington, DC, USA, 2004, pp. 21–30, doi: 10.1145/1030083.1030088.

[23] F. Erlacher and F. Dressler, "On high-speed flow-based intrusion detection using snort-compatible signatures," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 495–506, Jan./Feb. 2022.

[24] H. Gascon, A. Orfila, and J. B. Alıś, "Analysis of update delays in signature-based network intrusion detection systems," *Comput. Secur.*, vol. 30, no. 8, pp. 613–624, 2011, doi: 10.1016/j.cose.2011.08.010.

[25] T. Wang, X. Xie, W. Wang, C. Wang, Y. Zhao, and Y. Cui, "Netmamba: Efficient network traffic classification via pre-training unidirectional Mamba," in *Proc. 32nd IEEE Int. Conf. Netw. Protoc.*, Charleroi, Belgium, 2024, pp. 1–11, doi: 10.1109/ICNP61940.2024.10858569.

[26] Z. Zhao, Z. Li, Z. Song, W. Li, and F. T. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proc. ACM Web Conf.*, 2024, pp. 1608–1619.

[27] D. D. Monda, G. Bovenzi, A. Montieri, V. Persico, and A. Pescapè, "Analyzing the impact of shifts in encrypted mobile-app traffic on multimodal few-shot learning," *Comput. Netw.*, vol. 276, 2026, Art. no. 111935.

[28] C. Wu, J. Sun, J. Chen, M. Alazab, Y. Liu, and Y. Xiang, "TCG-IDS: Robust network intrusion detection via temporal contrastive graph learning," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 1475–1486, 2025.

[29] J. Gu and S. Lu, "An effective intrusion detection approach using SVM with naïve Bayes feature embedding," *Comput. Secur.*, vol. 103, 2021, Art. no. 102158, doi: 10.1016/j.cose.2020.102158.

[30] B. Anderson and D. A. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proc. 23 rd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, Halifax, NS, Canada, 2017, pp. 1723–1732, doi: 10.1145/3097983.3098163.

[31] O. Barut, M. Grohotolski, C. DiLeo, Y. Luo, P. Li, and T. Zhang, "Machine learning based malware detection on encrypted traffic: A comprehensive performance study," in *Proc. 7th Int. Conf. Netw., Syst. Secur.*, Dhaka, Bangladesh, 2020, pp. 45–55, doi: 10.1145/3428363.3428365.

[32] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "FS-Net: A flow sequence network for encrypted traffic classification," in *Proc. IEEE Conf. Comput. Commun.*, Paris, France, 2019, pp. 1171–1179, doi: 10.1109/INFOCOM.2019.8737507.

[33] Y. Liu, X. Wang, B. Qu, and F. Zhao, "ATVITSC: A novel encrypted traffic classification method based on deep learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 9374–9389, 2024, doi: 10.1109/TIFS.2024.3433446.

[34] X. Han, S. Liu, J. Liu, B. Jiang, Z. Lu, and B. Liu, "ECNet: Robust malicious network traffic detection with multi-view feature and confidence mechanism," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 6871–6885, 2024. [Online]. Available: https://doi.org/10.1109/TIFS.2024.3426304

[35] C. Yang et al., "Few-shot encrypted traffic classification via multi-task representation enhanced meta-learning," *Comput. Netw.*, vol. 228, 2023, Art. no. 109731.

[36] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[37] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. 25th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, 2018, pp. 1–15. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_03A-3_Mirsky_paper.pdf

[38] M. Catillo, A. Pecchia, and U. Villano, "CPS-GUARD: Intrusion detection for cyber-physical systems and IoT devices using outlier-aware deep autoencoders," *Comput. Secur.*, vol. 129, 2023, Art. no. 103210, doi: 10.1016/j.cose.2023.103210.

[39] S. Dou, K. Yang, Y. Jiao, C. Qiu, and K. Ren, "Anomaly detection in event-triggered traffic time series via similarity learning," *IEEE Trans. Dependable Secur. Comput.*, vol. 22, no. 2, pp. 888–902, Mar./Apr. 2025, doi: 10.1109/TDSC.2024.3418906.

[40] I. Nevat et al., "Anomaly detection and attribution in networks with temporally correlated traffic," *IEEE/ACM Trans. Netw.*, vol. 26, no. 1, pp. 131–144, Feb. 2018, doi: 10.1109/TNET.2017.2765719.

[41] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, San Diego, CA, USA, 2020, pp. 1–18.

[42] X. Yang, S. Ruan, J. Li, Y. Yue, and B. Sun, "TrafCL: Robust encrypted malicious traffic detection via contrastive learning," in *Proc. 33 rd ACM Int. Conf. Inf. Knowl. Manage.*, 2024, pp. 2910–2919.
[43] H. Wang, Y. Wang, Z. Gu, and Y. Jia, "AGAE: Unsupervised anomaly detection for encrypted malicious traffic," in *Proc. Web Big Data 8th Int. Joint Conf.*, Jinhua, China, 2024, pp. 448–464, doi: 10.1007/978-981-97-7241-4_28.
[44] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. 30th Annu. Netw. Distrib. Syst. Secur. Symp.*, San Diego, California, USA, 2023, pp. 2972–2987, doi: 10.14722/ndss.2023.23080.
[45] M. Wang, N. Yang, D. H. Gunasinghe, and N. Weng, "On the robustness of ML-based network intrusion detection systems: An adversarial and distribution shift perspective," *Computers*, vol. 12, no. 10, 2023, Art. no. 209.
[46] Y. Sharon, D. Berend, Y. Liu, A. Shabtai, and Y. Elovici, "TANTRA: Timing-based adversarial network traffic reshaping attack," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 3225–3237, 2022.
[47] Y. Tian, C. Zhang, Z. Guo, X. Zhang, and N. V. Chawla, "Learning MLPs on graphs: A unified view of effectiveness, robustness, and efficiency," in *Proc. 11th Int. Conf. Learn. Representations*, Kigali, Rwanda, 2023, pp. 1–12. [Online]. Available: https://openreview.net/forum?id=Cs3r5KLdoj
[48] X. Wang et al., "How powerful are spectral graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, Baltimore, Maryland, USA, 2022, pp. 23341–23362. [Online]. Available: https://proceedings.mlr.press/v162/wang22am.html
[49] Y. Tian, C. Zhang, Z. Guo, X. Zhang, and N. V. Chawla, "Learning MLPs on graphs: A unified view of effectiveness, robustness, and efficiency," in *Proc. 11th Int. Conf. Learn. Representations*, Kigali, Rwanda, 2023. [Online]. Available: https://openreview.net/forum?id=Cs3r5KLdoj
[50] C. Liu et al., "Where to mask: Structure-guided masking for graph masked autoencoders," in *Proc. 33rd Int. Joint Conf. Artif. Intell.*, 2024, pp. 2180–2188, doi: 10.24963/ijcai.2024/241.
[51] J. Li et al., "What's behind the mask: Understanding masked graph modeling for graph autoencoders," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, Long Beach, CA, USA, 2023, pp. 1268–1279, doi: 10.1145/3580305.3599546.
[52] Y. Cai, J. Zuo, M. Fan, C. Zhao, and Y. Lu, "An intrusion detection system for the CAN bus based on locality-sensitive hashing," *Electronics*, vol. 14, no. 13, 2025, Art. no. 2572. [Online]. Available: https://www.mdpi.com/2079-9292/14/13/2572
[53] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. 2017 Int. Conf. Inf. Netw.*, 2017, pp. 712–717.
[54] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. Inf. Syst. Secur. Privacy*, 2018, pp. 108–116. [Online]. Available: https://api.semanticscholar.org/CorpusID:4707749
[55] M. C. P. Saheb, M. S. Yadav, S. Babu, J. J. Pujari, and J. B. Maddala, "A review of DDoS evaluation dataset: Cicddos2019 dataset," in *Energy Systems, Drives and Automations*, J. R. Szymanski, C. K. Chanda, P. K. Mondal, and K. A. Khan Eds. Berlin, Germany: Springer, 2023, pp. 389–397.
[56] S. Garcıá, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and legitimate IoT network traffic (version 1.0.0) [data set]," 2020. [Online]. Available: https://doi.org/10.5281/zenodo.4743746

Junbo Jia received the bachelor's degree from Xidian University, Xi'an, China. He is currently working toward the PhD degree with the School of Computer Science and Technology, Xidian University, Xi'an, China. His current research interests include intrusion detection and network security.

Ao Wang is currently working toward the master's degree with the School of Computer Science and Technology, Xidian University, Xi'an, China. His current research interests include encrypted traffic detection.

Li Yang (Member, IEEE) received the BS degree in instructional technology from Shaanxi Normal University, in 1999, the MS degree in computer science from Xidian University, in 2005, and the PhD degree in cryptography from Xidian University, Xi'an, China, in 2010. He was a visiting postdoctoral researcher in the Complex Networks & Security Research (CNSR) Lab, Virginia Tech from 2013 to 2014. He is currently a professor in School of Computer Science and Technology, Xidian University. His research interests include applied cryptography, wireless network security, cloud computing security, and trusted computing.

Lu Zhou (Member, IEEE) received the PhD degree in computer science and technology from Shanghai Jiao Tong University, in 2022. He is an associate professor with the School of Computer Science and Technology, Xidian University. His research interests include IoT security, data security and privacy protection.

Huipeng Yang received the bachelor's degree from Zhengzhou University, Xi'an, China. He is currently working toward the PhD degree with the School of Computer Science and Technology, Xidian University, Xi'an, China. His current research interests include cyberspace mapping and cyber security.

Anyuan Sang received the bachelor's degree from the Xi'an University of Science and Technology, Xi'an, China. He is currently working toward the PhD degree with the School of Computer Science and Technology, Xidian University, Xi'an, China. His current research interests include intrusion detection and APT detection.